Informe del Projecte Neo4j

Padró Històric

Marc Dalmau Guamis - 1710713
Alejandro Flores Hernández - 1709458
Javier Martínez Hernández - 1708408
Dídac Tresserres Saló - 1703771

PART 1 **3**

1. Introducció i objectius 3
2. Disseny del model 3
3. Implementació i càrrega de dades 4
    3.1. Constraints i índexs 4
    3.2. Script de càrrega 4
4. Anàlisi i consultes 5

PART 2 **10**

1. Propostes de consultes 10
2. Disseny del nou model 12
3. Implementació i càrrega de dades 13
    3.1. Constraints i índexs 13
    3.2. Script de càrrega 14
4. Consultes amb el nou model 15
5. Anàlisi de rendiment 17
6. Avantatges i inconvenients 18

PART 3 **19**

TREBALL EN EQUIP **23**

PART 1

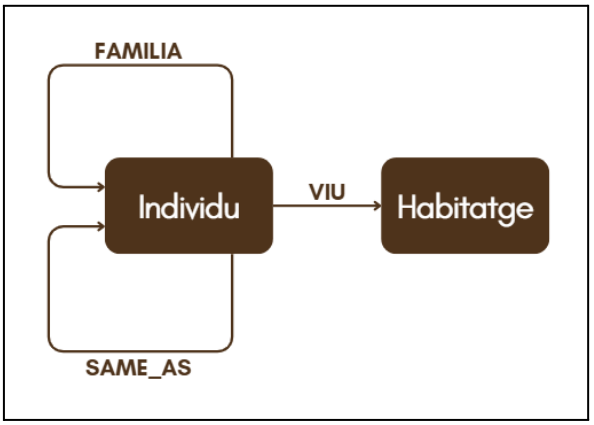
1. Introducció i objectius

L'objectiu d'aquesta primera fase és realitzar la transició d'un model de dades relacional a un model orientat a grafs utilitzant Neo4j. Aquest projecte busca explotar les relacions complexes entre individus i la seva evolució temporal en els padrons històrics de Sant Feliu de LLobregat i Castellví de Rosanes.

2. Disseny del model

A diferència del disseny en MongoDB, on l'estructura era més estàtica, el model en Neo4j s'ha dissenyat per optimitzar la traçabilitat, creant un total de 2 nodes (*individu i habitatge*) i 3 relacions (*viu, família i same_as*).

- **INDIVIDU:** Contenen la informació personal (noms i cognoms).
- **HABITATGE:** Representen la unitat convivencial en un lloc i any concrets.
- **VIU:** Connecta cada individu amb el seu habitatge/padró.
- **FAMILIA**: Defineix els vincles de parentiu amb el/la cap de família.
- **SAME_AS:** Connecta els identificadors d'empadronament corresponents a un mateix individu. Aquesta és la clau del projecte, ja que permet unir diferents registres d'un mateix individu a través dels anys, solucionant variacions en l'escriptura dels noms.



3. Implementació i càrrega de dades

Per la ingesta de dades des dels fitxers CSV hem creat el fitxer *importacio_part1.cypher*, clau per assegurar la consistència del graf.

3.1. Constraints i índexs

S'han definit regles per optimitzar el rendiment de les consultes i evitar errors en la inserció de dades.

Hem definit 2 constraints d'unicitat, una per cada tipus de node. Per a *Individu* hem establert l'atribut *individuID* com a clau de node, mentre que per a *Habitatge* hem definit una clau composta pels atributs *llarID*, *anyPadro* i *location*.

```
CREATE CONSTRAINT UniqueIndividu IF NOT EXISTS FOR (i:Individu) REQUIRE i.individuId IS NODE KEY;
```

```
CREATE CONSTRAINT UniqueHabitatge IF NOT EXISTS FOR (h:Habitatge) REQUIRE (h.llarId, h.anyPadro, h.location) IS NODE KEY;
```

També hem definit 3 índexs sobre els camps *anyPadro, nom + cognoms* i *relacioHarmonitzada*, amb l'objectiu d'accelerar les consultes de filtratge demogràfic i cerques nominals demanades al següent apartat.

```
CREATE INDEX NomCognomComplet IF NOT EXISTS FOR (i:Individu) ON (i.name, i.surname, i.secondSurname);
```

```
CREATE INDEX AnyHabitatge IF NOT EXISTS FOR (h:Habitatge) ON (h.anyPadro);
```

```
CREATE INDEX RelacioHarmonitzada IF NOT EXISTS FOR ()-[r:FAMILIA]-() ON (r.relacioHarmonitzada);
```

3.2. Script de càrrega

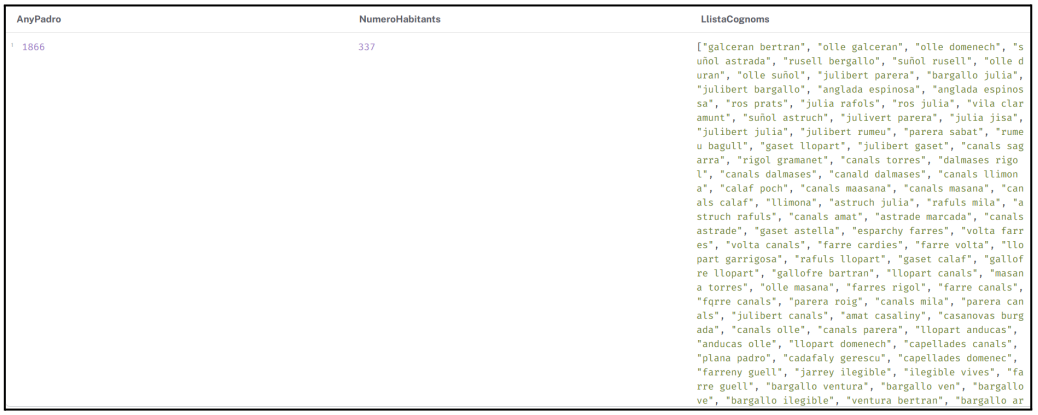
Al fitxer amb el que carreguem les dades hem utilitzat *LOAD CSV* amb transformacions del tipus *toInteger* i filtratge de valors nuls. La càrrega la dividim en dues etapes, primer els nodes i després les relacions. D'aquesta manera assegurem que els punts d'unió existents estiguin ben definits.

4. Anàlisi i consultes

En aquest apartat mostrarem les consultes realitzades per respondre les preguntes de l'exercici 2, proporcionant el codi i una breu explicació. L'objectiu és demostrar la capacitat del model per resoldre problemes de neteja de dades, relacions convivencials, traçabilitat d'individus i càlculs estadístics.

a) Per a cada padró (any) de Castellví de Rosanes, retorna l'any de padró, el número d'habitants, i la llista de cognoms. Elimina duplicats i "nan".

```
MATCH (i:Individu)-[:VIU]->(h:Habitatge)
WHERE h.location = "CR"
WITH h.anyPadro AS Any, i,
    CASE
      WHEN i.secondSurname IS NULL OR i.secondSurname = "nan" OR i.secondSurname = ""
      THEN i.surname
      ELSE i.surname + " " + i.secondSurname
    END AS CognomNet
WHERE CognomNet IS NOT NULL AND CognomNet <> "nan" AND CognomNet <> ""
RETURN Any AS AnyPadro, count(DISTINCT i) AS NumeroHabitants, collect(DISTINCT CognomNet)
AS LlistaCognoms
ORDER BY AnyPadro
```



AnyPadro	NumeroHabitants	LlistaCognoms
1866	337	["galceran bertran", "olle galceran", "olle domenech", "suñol astrada", "rusell bergallo", "suñol rusell", "olle duran", "olle suñol", "julibert parera", "bargallo julia", "julibert bargallo", "anglada espinosa", "anglada espinosa", "ros prats", "julia rafols", "ros julia", "vila clar amunt", "suñol astruch", "julivert parera", "julia jisa", "julibert julia", "julibert rumeu", "parera sabat", "rumeu bagull", "gaset llopart", "julibert gaset", "canals sagarra", "rigol gramanet", "canals torres", "dalmases rigol", "canals dalmases", "canald dalmases", "canals llimona", "calaf poch", "canals maasana", "canals masana", "canals calaf", "llimona", "astruch julia", "rafuls mila", "astruch rafuls", "canals amat", "astrade marcada", "canals astrade", "gaset astella", "esparchy farres", "volta farres", "volta canals", "farre cardies", "farre volta", "llopart garrigosa", "rafuls llopart", "gaset calaf", "gallofre llopart", "gallofre bartran", "llopart canals", "masana torres", "olle masana", "farres rigol", "farre canals", "fqrre canals", "parera roig", "canals mila", "parera canals", "julibert canals", "amat casaliny", "casanovas burgada", "canals olle", "canals parera", "llopart anducas", "anducas olle", "llopart domenech", "capellades canals", "plana padro", "cadafaly gerescu", "capellades domenec", "farreny guell", "jarrey ilegible", "ilegible vives", "farre guell", "bargallo ventura", "bargallo ven", "bargallo ve", "bargallo ilegible", "ventura bertran", "bargallo ar

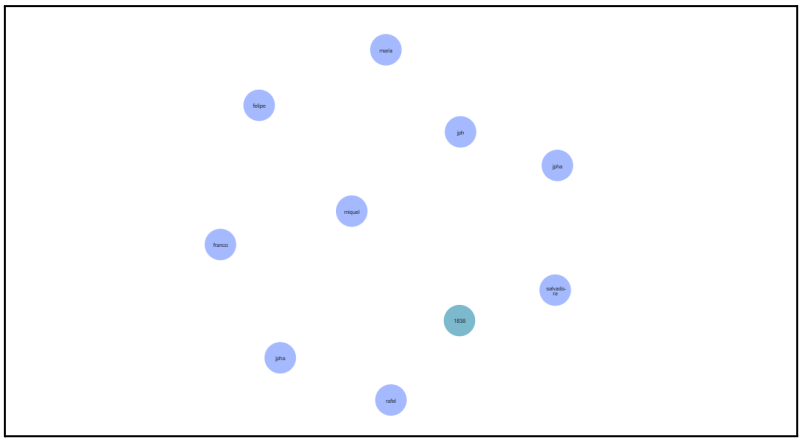
Per resoldre aquesta consulta hem utilitzat una clàusula *CASE* que ens permet netejar les dades en temps d'execució. Si un individu té un "nan" o un valor buit en el seu segon cognom, el sistema només agafa el primer. Després, utilitzem *count(DISTINCT i)* per comptar persones úniques i collect(DISTINCT) per agrupar els cognoms en una llista sense repeticions per cada any.

b) Retorna el nom de les persones que vivien al mateix habitatge que "rafel marti" (no té segon cognom) segons el padró de 1838 de Sant Feliu de Llobregat (SFLL). Retorna la informació en mode graf i mode llista.

```
//Mode llista
MATCH (rafel:Individu {name: "rafel", surname: "marti"})-[:VIU]->(h:Habitatge {anyPadro: 1838,
location: "SFLL"})
MATCH (altre:Individu)-[:VIU]->(h)
WHERE altre <> rafel
RETURN altre.name AS Nom;
```

Nom
1 "salvadora"
2 "jpha"
3 "jph"
4 "maria"
5 "felipe"
6 "miquel"
7 "franco"
8 "jpha"

```
//Mode graf
MATCH (rafel:Individu {name: "rafel", surname: "marti"})-[:VIU]->(h:Habitatge {anyPadro: 1838,
location: "SFLL"})<-[:VIU]-(altre:Individu)
RETURN rafel, h, altre;
```



Primer localitzem el node de “Rafel Martí” i el seu habitatge del padró de 1838. Un cop situats a l’habitatge, busquem tots els altres nodes d’individu que tinguin una relació *[:VIU]* cap aquest mateix node. Finalment, retornem el resultat en mode llista i en mode graf tal com diu l’enunciat.

c) Retorna totes les aparicions de "miguel estape bofill". Fes servir la relació SAME_AS per poder retornar totes les instancies, independentment de si hi ha variacions lèxiques (ex. diferents formes d'escriure el seu nom/cognoms). Mostra la informació en forma de taula: el nom, la llista de cognoms i la llista de segon cognom (elimina duplicats).

```
MATCH (i1:Individu)-[:SAME_AS*0..]-(i2:Individu)
WHERE i1.name = "miguel" AND i1.surname = "estape" AND i1.secondSurname = "bofill"
RETURN i2.name AS Nom,
    collect(DISTINCT i2.surname) AS Cognoms,
    collect(DISTINCT i2.secondSurname) AS SegonsCognoms;
```

Nom	Cognoms	SegonsCognoms
"miguel"	["estape"]	["bofill", "bufill"]

Com hem dit anteriorment, la relació *same_as* ens permet connectar registres que es refereixen a la mateixa persona física en diferents anys. D'aquesta manera, utilitzant l'operador de camí variable **0..* , buquem totes les connexions possibles per trobar totes les variants del nom o cognoms que hagin estat vinculades a aquest individu.

d) Mitja de fills a Sant Feliu de Llobregat l'any 1881 per família. Fes servir CALL per obtenir el nombre de llars.

```
CALL () {
   MATCH (h:Habitatge)
   WHERE h.location = "SFLL" AND h.anyPadro = 1881
   RETURN count(DISTINCT h) AS TotalHabitatges
}

MATCH (h:Habitatge {location: "SFLL", anyPadro: 1881})<-[:VIU]-(i1:Individu)
MATCH (i1)-[rel:FAMILIA]->(i2:Individu)
WHERE rel.relacioHarmonitzada IN ["fill", "filla"]
WITH count(DISTINCT i2) AS TotalFills, TotalHabitatges
RETURN TotalFills,
    TotalHabitatges,
    toFloat(TotalFills) / TotalHabitatges AS MitjaFillsPerHabitatge;
```

TotalFills	TotalHabitatges	MitjaFillsPerHabitatge
1289	596	2.162751677852349

Aquí utilitzem *CALL* per encapsular una subconsulta que compta el total d'habitatges únics en l'any seleccionat. Un cop tenim aquest valor fix *(TotalHabitatges)*, busquem les relacions de *família* de tipus *"fill/filla"* en aquests habitatges. Finalment, dividim el total de fills entre el total d'habitatges per obtenir la mitjana.

e) Mostreu les famílies de Castellví de Rosanes amb més de 3 fills. Mostreu el nom i cognoms del cap de família i el nombre de fills. Ordeneu-les pel nombre de fills fins a un límit de 20.

```
MATCH (cap:Individu)-[:VIU]->(h:Habitatge)
WHERE h.location = "CR"
MATCH (cap)-[rel:FAMILIA]->(fill:Individu)
WHERE rel.relacioHarmonitzada in ["fill", "filla"]
WITH cap, count(DISTINCT fill) AS TotalFills
WHERE TotalFills > 3
RETURN cap.name AS NomCap, cap.surname AS CognomCap, cap.secondSurname AS
SegonCognomCap, TotalFills
ORDER BY TotalFills DESC
LIMIT 20;
```

	NomCap	CognomCap	SegonCognomCap	TotalFills
1	"pablo"	"astruch"	"julia"	7
2	"jose"	"olle"	"domenech"	6
3	"benito"	"julivert"	"parera"	6
4	"jose"	"canals"	"olle"	6
5	"pedro"	"bargallo"	"ilegible"	6
6	"jose"	"canals"	"mila"	6
7	"jose"	"rafuls"	"mila"	5
8	"jaime"	"jarrey"	"ilegible"	5
9	"pablo"	"bargallo"	"armangol"	5
10	"francisco"	"aregay"	"rigol"	5

	NomCap	CognomCap	SegonCognomCap	TotalFills
11	"ramon"	"canals"	"amat"	4
12	"pablo"	"canals"	"llimona"	4
13	"jaime"	"gallofre"	"bartran"	4
14	"tomas"	"parera"	"roig"	4
15	"juan"	"julibert"	"parera"	4
16	"jose"	"llopart"	"pareyada"	4
17	"jose"	"suñol"	"henan"	4
18	"estevan"	"gallofre"	"bertran"	4
19	"cristobal"	"olle"	"rabantos"	4
20	"pedro"	"farres"	"rigol"	4

Per identificar el *"cap de familia"*, busquem individus que tinguin una relació de parentiu cap a altres individus identificats com a fills. Utilitzem *count(DISTINCT fill)* per quantificar la descendència de cada cap i filtrem amb un *WHERE* per quedar-nos només amb aquells que superen el llindar 3.

f) Per cada padró/any de Sant Feliu de Llobregat, mostra el carrer amb menys habitants i el nombre d'habitants en aquell carrer. Fes servir la funció min() i CALL.

```
MATCH (i:Individu)-[:VIU]->(h:Habitatge)
WHERE h.location = "SFLL"
WITH h.anyPadro AS AnyPadro, h.carrer AS Carrer, count(i) AS HabitantsPerCarrer
CALL (AnyPadro) {
   MATCH (i2:Individu)-[:VIU]->(h2:Habitatge)
   WHERE h2.location = "SFLL" AND h2.anyPadro = AnyPadro
   WITH h2.carrer AS Carrer2, count(i2) AS Habitants2
   RETURN min(Habitants2) AS MinHabitants
}
WITH AnyPadro, Carrer, HabitantsPerCarrer, MinHabitants
WHERE HabitantsPerCarrer = MinHabitants
RETURN AnyPadro, Carrer, HabitantsPerCarrer AS TotalHabitants
ORDER BY AnyPadro
```

	AnyPadro	Carrer	TotalHabitants
1	1833	"carrtera de la part de molins de rey"	5
2	1838	"carretera de barna"	30
3	1839	"casas del 3onmany"	3
4	1878	"carrretera"	2
5	1881	"Carretera"	5
6	1889	"s n antonio"	1

Primer comptem els habitants per cada combinació d'any i carrer. Després, mitjançant *CALL (AnyPadro)*, executem una subconsulta per a cada any que troba quinés el valor mínim de persones en qualsevol carrer d'aquell any concret. Finalment, filtrem els resultats per mostrar només els carrers que coincideixen amb aquest valor mínim.

PART 2

Aquesta part de la pràctica correspon a l'exercici 3, el qual ens demanava dissenyar un altre esquema de la base de dades per tal d'optimitzar 2 consules. L'objectiu d'aquesta part és aplicar un refactoring òptim en funció de les consultes i exposar els avantatges i inconvenients d'aquest nou model.

1. Propostes de consultes

Hem dissenyat dues consultes inspirades en les de la part 1 però amb variacions pròpies. Ambdues pateixen el mateix problema estructural al model original, cal filtrar per propietats no indexades (*location* i *anyPadro*) sobre tots els nodes *Habitatge*, que és el nivell 3-4 de la jerarquia (vista a classe) d'eficiència de NEO4j.

Consulta 1 - Per a cada combinació de municipi i any de padró, mostra el total de fills, el nombre d'habitatges i la mitjana de fills per habitatge.

Aquesta consulta és una versió de l'apartat *d* de l'exercici 2, amb la diferència que aquella estava fixada al municipi de SFLL i a l'any 1881. En canvi, ara la consulta recorre tots els municipis i tots els anys de forma genèrica, i retorna una fila per cada combinació.

Amb el model original la consulta quedaria de la següent manera:

```
MATCH (h:Habitatge)<-[:VIU]-(i1:Individu)-[rel:FAMILIA]->(i2:Individu)
WHERE rel.relacioHarmonitzada IN ['fill', 'filla']
WITH h.location AS Municipi, h.anyPadro AS Any, count(DISTINCT i2) AS TotalFills
CALL (Municipi, Any) {
   MATCH (h2:Habitatge)
   WHERE h2.location = Municipi AND h2.anyPadro = Any
   RETURN count(DISTINCT h2) AS TotalHabitatges
}
RETURN Municipi, Any, TotalFills, TotalHabitatges, toFloat(TotalFills) / TotalHabitatges AS MitjaFills
ORDER BY Municipi, Any;
```

El principal problema d'aquesta consulta és que el *MATCH* inicia escanejant tots els nodes *Habitatge* sense cap punt d'entrada indexat (nivell 3-4) i, a més, el *CALL* intern repeteix un filtre per propietats (*location*, *anyPadro*) per cada combinació (municipi,any), multiplicant el cost.

<u>Consulta 2</u> - Mostra les esposes que van canviar de residència dins del municipi SFLL en dos padrons consecutius. Per cada cas, mostra el nom i cognom de l'esposa, l'any del padró anterior, el carrer on vivia, l'any del padró posterior i el carrer nou.

En aquest cas, la consulta és una proposta pròpia. Hem cregut que era un bon enunciat per optimitzar el model original.

Amb el model original la consulta quedaria de la següent manera:

```
MATCH (i1:Individu)-[rel:FAMILIA]->(esposa:Individu)
WHERE rel.relacioHarmonitzada = 'esposa'
MATCH (esposa)-[:SAME_AS*]-(altra:Individu)
WITH DISTINCT esposa, altra
MATCH (esposa)-[:VIU]->(h1:Habitatge)
MATCH (altra)-[:VIU]->(h2:Habitatge)
WHERE h1.anyPadro < h2.anyPadro AND h1.location = 'SFLL' AND h2.location = 'SFLL'
    AND (h1.carrer <> h2.carrer OR h1.numero <> h2.numero)
    AND NOT EXISTS {
        MATCH (esposa)-[:SAME_AS*]-(intermedia:Individu)-[:VIU]->(hMig:Habitatge)
        WHERE hMig.location = 'SFLL' AND h1.anyPadro < hMig.anyPadro
                AND hMig.anyPadro < h2.anyPadro
    }
RETURN DISTINCT esposa.name AS Nom, esposa.surname AS Cognom,
          h1.anyPadro AS AnyAnterior, h1.carrer AS CarrerAnterior, h1.numero AS NumeroAnterior,
          h2.anyPadro AS AnyPosterior, h2.carrer AS CarrerPosterior, h2.numero AS NumeroPosterior
ORDER BY h2.anyPadro, Cognom;
```

El problema en aquesta consulta el trobem en que el punt d'entrada és un scan de totes les relacions *família* per filtrar les que són esposa, que representa el nivell 5 (el

pitjor). A més, després es filtren *h1.anyPadro* i *h1.location* sobre els Habitatges, que representa els nivells 3-4. És a dir, ens trobem tres nivells molt baixos encadenats.

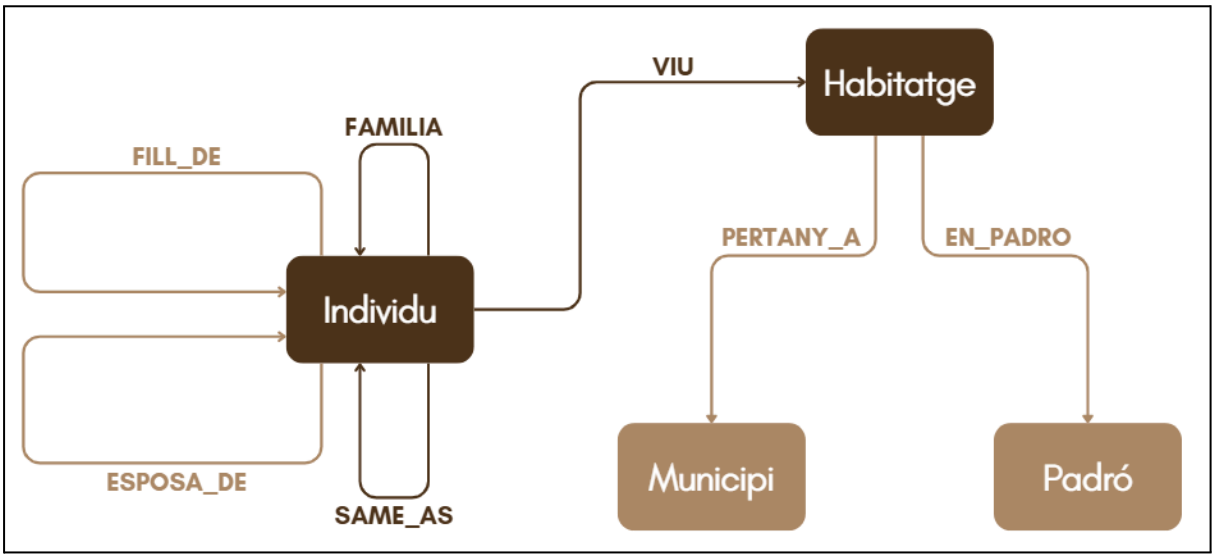
2. Disseny del nou model

El nou model que proposem separa els conceptes de municipi i padró en nodes independents i extreu les relacions de parentesc més consultades com a tipus propis. Això permet que les consultes comencin des d'un node ancla amb propietat indexada, que representa els nivells 1-2, el màxim d'eficiència en Neo4j.

Amb els canvis implementats ens queda un disseny de 4 nodes (*individu, habitatge, padró i municipi*) i 7 relacions (*viu, pertany_a, en_padro, familia, fill_de, esposa_de i same_as*).

- **INDIVIDU:** Contenen la mateixa informació que abans (noms i cognoms).
- **HABITATGE:** En aquest cas, hem hagut conservar totes les dades que tenia el model original ja que l'identificador de cada llar no és únic. A la base de dades ens trobem que un mateix id d'una casa es reutilitza en diferents municipis i anys.
- **PADRÓ:** Representen un any de padró i només contenen l'atribut d'any.
- **MUNICIPI:** Contenen el nom del municipi en qüestió.
- **VIU:** Igual que abans, connecta cada individu amb el seu habitatge.
- **PERTANY_A:** Connecta cada habitatge amb el municipi al qual pertany.
- **EN_PADRO:** Uneix cada habitatge amb l'any de padró que es va registrar.
- **FAMILIA**: La mateixa relació que abans, defineix els vincles entre el/la cap de família, però exceptuant als fills i a les esposes.
- **FILL_DE:** És un subconjunt de la relació de *família* del model original, on només unim als caps de família amb els seus fills.
- **ESPOSA_DE:** La mateixa relació que *fill_de* però unint els caps de família amb les seves respectives esposes.
- **SAME_AS**: Idèntica al model original però amb una reducció transitiva aplicada.

Així doncs, l'esquema del nou model quedaria de la següent manera:



3. Implementació i càrrega de dades

Per importar les dades d'aquest nou model hem creat un nou fitxer (*script_neo4j_Part2.cypher*), que fa servir els mateixos fitxers CSV que abans però amb l'estructura que acabem de definir.

3.1. Constraints i índexs

En aquest cas hem definit 4 constraints d'unicitat, una per cada tipus de node. Tant per a *Individu* com per a *Habitatge* hem conservat els dos constraints de clau node sobre els mateixos atributs, mentre que pels nous nodes dels tipus *Padró* i *Municipi* hem creat dos constraints d'unicitat. Pel primer hem establert l'atribut *anyPadro* com a únic i pel segon l'atribut *municipi*, també com a únic.

```
CREATE CONSTRAINT UniqueIndividu IF NOT EXISTS FOR (i:Individu) REQUIRE i.individuId IS NODE KEY;
```

```
CREATE CONSTRAINT UniqueHabitatge IF NOT EXISTS FOR (h:Habitatge) REQUIRE (h.llarId, h.anyPadro, h.location) IS NODE KEY;
```

```
CREATE CONSTRAINT UniquePadro IF NOT EXISTS FOR (pa:Padro) REQUIRE pa.anyPadro IS UNIQUE;

CREATE CONSTRAINT UniqueMunicipi IF NOT EXISTS FOR (m:Municipi) REQUIRE m.municipi IS UNIQUE;
```

No obstant, en aquest model no hem definit cap índex ja que els que necessitem ja es generen automàticament amb la creació dels constraints anteriors.

3.2. Script de càrrega

El fitxer amb el que carreguem les dades és semblant al del model original. L'estructura és la mateixa, primer carreguem tots els nodes (4) i després les relacions (7). Les modificacions les trobem en la creació dels nous nodes i relacions, la subdivisió de la relació *família* i l'optimització de la relació *same_as*.

Pels nodes i relacions dels habitatges és fàcil, només creem un node nou per cada any de padró o municipi diferent que hi ha a la base de dades i el vinculem amb el seu respectiu habitatge.

Per altra banda, per la relació de *família* vam implementar 3 blocs CALL independents per, abans de crear la relació, saber si era el cas de fill/a, esposa o qualsevol altre.

Per últim, a la relació *same_as*, com hem mencionat anteriorment, hem aplicat una reducció transitiva. En la base de dades original, quan un mateix individu apareix en múltiples padrons amb identificadors diferents, es creaven relacions *same_as* innecessàries. Per exemple, si un individu té 3 identificadors (A, B i C):

$$A \rightarrow B \ , \ B \rightarrow C \ , \ A \rightarrow C$$

siguent aquest última relació redundant ja que, en aquest cas, només en calen 2 perquè $A \rightarrow C$ és deduïble per transitivitat a través de $A \rightarrow B \rightarrow C$. Així doncs, aplicant la reducció transitiva, el graf queda en la cadena mínima:

$$A \rightarrow B \rightarrow C$$

4. Consultes amb el nou model

<u>Consulta 1</u> - Mitjana de fills per habitatge, per municipi i any

```
MATCH (m:Municipi)<-[:PERTANY_A]-(h:Habitatge)-[:EN_PADRO]->(pa:Padro)
MATCH (h)<-[:VIU]-(i1:Individu)<-[:FILL_DE]-(i2:Individu)
WITH m, pa, count(DISTINCT i2) AS TotalFills
CALL (m, pa) {
   MATCH (h2:Habitatge)-[:PERTANY_A]->(m)
   MATCH (h2)-[:EN_PADRO]->(pa)
   RETURN count(DISTINCT h2) AS TotalHabitatges
}
RETURN m.municipi AS Municipi, pa.anyPadro AS Any, TotalFills, TotalHabitatges,
          toFloat(TotalFills) / TotalHabitatges AS MitjaFills
ORDER BY Municipi, Any;
```

Aquesta consulta aprofita diverses millores del nou model. En primer lloc, el punt d'entrada és el node *Municipi*, que disposa d'un constraint d'unicitat i, per tant, d'un índex automàtic. Això permet localitzar el municipi de forma directa sense escanejar cap altre node del graf (nivell 1-2).

En segon lloc, la identificació dels fills es fa mitjançant la relació *fill_de* (nivell 2), quan en el model anterior aquesta mateixa operació requeria recórrer totes les relacions *família* i aplicar un filtre sobre la propietat *relacioHarmonitzada* (nivell 5). D'aquesta manera ens estalviem una operació molt costosa ja que les propietats no estan indexades per defecte.

Finalment, el bloc CALL rep els nodes *municipi* i *padró* com a referències concretes ja resoltes en les línies anteriors de la consulta. Això evita que Neo4j hagi de tornar a localitzar-los dins del subquery, eliminant qualsevol scan addicional i fent que el recompte d'habitatges per municipi i any sigui una operació purament local sobre nodes ja coneguts.

<u>Consulta 2</u> - Esposes registrades a "SFLL" que van canviar de residència entre padrons consecutius.

```
MATCH (esposa:Individu)-[:ESPOSA_DE]->(marit:Individu)
MATCH (esposa)-[:SAME_AS*]-(altra:Individu)
WITH DISTINCT esposa, altra
MATCH (esposa)-[:VIU]->(h1:Habitatge)-[:PERTANY_A]->(:Municipi {municipi: 'SFLL'})
MATCH (altra)-[:VIU]->(h2:Habitatge)-[:PERTANY_A]->(:Municipi {municipi: 'SFLL'})
WHERE h1.anyPadro < h2.anyPadro AND (h1.carrer <> h2.carrer OR h1.numero <> h2.numero)
   AND NOT EXISTS {
      MATCH (esposa)-[:SAME_AS*]-(intermedia:Individu)-[:VIU]->(hMig:Habitatge)-[:PERTANY_A]->
            (:Municipi {municipi: 'SFLL'})
      WHERE hMig.anyPadro > h1.anyPadro AND hMig.anyPadro < h2.anyPadro
            AND intermedia <> esposa AND intermedia <> altra
   }
RETURN DISTINCT esposa.name AS Nom, esposa.surname AS Cognom, h1.anyPadro AS
      AnyAnterior, h1.carrer AS CarrerAnterior, h1.numero AS NumeroAnterior, h2.anyPadro AS
       AnyPosterior, h2.carrer AS CarrerNou, h2.numero AS NumeroNou
ORDER BY h2.anyPadro, Cognom;
```

Aquesta consulta també es beneficia de les millores del nou model. En primer lloc, el punt d'entrada és la relació *esposa_de* (nivell 2), quan en el model anterior, al igual que a la consulta anterior, calia recórrer totes les relacions *família* (nivell 5).

En segon lloc, la localització geogràfica de les esposes ja no es fa filtrant per la propietat *location* = 'SFLL' directament sobre els habitatges, sinó navegant fins al node Municipi amb constraint d'unicitat. Això permet que Neo4j localitzi el municipi per índex i després travessi les relacions *pertany_a* en sentit invers per obtenir els habitatges associats, en lloc d'escanejar tots els habitatges del graf i comprovar la propietat un per un.

Aquesta mateixa millora s'aplica també dins del bloc NOT EXISTS, on la subquery també filtra per municipi. Gràcies al node Municipi indexat, el subquery no requereix cap scan addicional per localitzar el context geogràfic, mantenint el cost de la comprovació d'existència al mínim possible.

5. Anàlisi de rendiment

CONSULTA 1			
Mètrica	**Model Original**	**Nou Model**	**Millora**
Total database accesses (db hits)	108.447	76.885	-29,1%
Total allocated memory	145.296 bytes	144.616 bytes	similar

El nou model aconsegueix una reducció del 29% en accessos a la base de dades amb un consum de memòria pràcticament idèntic. Malgrat la modesta reducció d'accessos, l'impacte en temps d'execució és molt superior, assolint una execució 22 vegades més ràpida que el model original.

CONSULTA 2			
Mètrica	**Model Original**	**Nou Model**	**Millora**
Total database accesses (db hits)	129.790	75.171	-42,1%
Total allocated memory	650.200 bytes	767.340 bytes	+18% més en memòria

El nou model aconsegueix una reducció del 42% en accessos a la base de dades. L'únic cost associat és un increment del 18% en memòria, degut als salts addicionals als nodes *Municipi* dins del NOT EXISTS, que queda àmpliament compensat per la millora en accessos a disc.

6. Avantatges i inconvenients

Les dades de rendiment de la secció anterior confirmen que el nou model representa una millora respecte a l'original. La consulta 1 redueix els accessos a base de dades un 29%, assolint una execució 22 vegades més ràpida amb un consum de memòria pràcticament idèntic. La consulta 2 va encara més lluny, reduint els accessos un 42%, amb l'únic cost d'un increment del 18% en memòria que queda àmpliament compensat per la dràstica reducció d'accessos a disc.

Aquests guanys s'expliquen per quatre decisions de disseny clau. En primer lloc, la introducció de nodes *Municipi* amb constraint d'unicitat permet entrar al graf per un lookup indexat de nivell 1-2 en lloc d'escanejar tots els habitatges i filtrar per propietat. En segon lloc, les relacions semàntiques *fill_de* i *esposa_de*, amb 4.556 i 2.285 instàncies respectivament extretes de *família*, eliminen completament la necessitat d'avaluar propietats per cada relació, passant d'una operació de nivell 5 a una de nivell 2. En tercer lloc, els nodes *Municipi* i *Padró* com a entitats pròpies eliminen la redundància semàntica del model original, on la mateixa informació es repetia en cada *Habitatge*, i permeten que el CALL intern rebi aquests nodes com a context ja resolt, evitant qualsevol re-scan addicional. Finalment, la reducció transitiva de *same_as* redueix el nombre de relacions al graf i simplifica tots els recorreguts que fan servir [:SAME_AS*].

El nou model presenta, però, alguns inconvenients que cal esmentar. Les consultes fan més salts encadenats, especialment la consulta 2, que navega per nodes *Municipi* tant a la consulta principal com dins del NOT EXISTS, cosa que explica l'increment del 18% en memòria. La càrrega de dades també s'ha complicat lleugerament, la importació de *família* ara requereix tres blocs CALL separats, quan en el model original es resolia en una sola operació. Això es tracta, però, d'un cost puntual que no afecta el rendiment de les consultes. A més, fills i esposes ja no formen part de *família*, de manera que qualsevol consulta del model original que filtri per fill, filla o esposa requeriria adaptació. Cal afegir també que el nou model no elimina completament la redundància d'emmagatzematge, els nodes *Habitatge* continuen necessitant *anyPadro* i *location* com a part de la seva clau composta, ja que amb el *llarId* sol no és possible identificar unívocament una llar.

PART 3

Aquesta última part de la pràctica correspon a l'exercici 4, on l'objectiu és aplicar algorismes d'analítica de grafs sobre les dades del model original (part 1) per extreure informació estructural que no és accessible mitjançant consultes Cypher convencionals. Concretament, analitzem la connectivitat del graf mitjançant components connexes i calculem la similitud entre nodes com a pas previ a la identificació d'individus que són la mateixa persona en padrons d'anys diferents.

Apartat A - Estudi de les components connexes i de l'estructura de les components.

Per analitzar l'estructura de connectivitat del graf apliquem l'algorisme Weakly Connected Components (wcc), que identifica subgrafs desconnectats dins la xarxa. Projectem en memòria els nodes *Individu* i *Habitatge* amb la relació *viu*.

```
CALL gds.graph.project(
  'grafCC',
  ['Individu', 'Habitatge'],
  'VIU'
)
```

Un cop projectat el graf, executem wcc en mode stream i realitzem dues consultes d'exploració per entendre l'estructura resultant.

1. Mostra, en forma de taula, les 10 components connexes més grans (ids i mida).

```
CALL gds.wcc.stream('grafCC')
YIELD componentId, nodeId
WITH componentId, size(collect(nodeId)) as mida
ORDER BY mida DESC
LIMIT 10
RETURN componentId AS ComponentID, mida as Mida
```

Aquesta consulta agrupa tots els nodes per component i retorna les deu amb més nodes. Cada component connexa representa un conjunt d'individus i habitatges interconnectats, és a dir, persones que viuen o han viscut en habitatges relacionats directa o indirectament. La component més gran concentra la majoria dels nodes del graf, cosa habitual en xarxes reals que segueixen una distribució de llei de potència, és a dir, unes poques components molt grans i moltes de petites o individuals. Les components de mida 1 corresponen a individus sense cap relació *viu* associada, és a dir, registres orfes sense habitatge assignat.

2. Per cada municipi i any, mostra el nombre de parelles del tipus: (Individu)—(Habitatge).

```
CALL gds.wcc.stream('grafCC')
YIELD nodeId, componentId
MATCH (n) WHERE id(n) = nodeId
WITH componentId, collect(n) AS nodes
UNWIND nodes AS n1
UNWIND nodes AS n2
WITH componentId, n1, n2
WHERE n1:Individu AND n2:Habitatge
MATCH (n1)-[:VIU]->(n2)
RETURN n2.location AS Municipi, n2.anyPadro AS Any, count(*) AS NombreParelles
ORDER BY Municipi, Any
```

Aquesta consulta explora la distribució de relacions *viu* per municipi i any de padró. Per cada component connexa, identifica les parelles formades per un individu i un habitatge connectats per la relació *viu*, i les agrupa per municipi i any. Els resultats proporcionen una visió de l'evolució temporal del nombre d'habitants registrats a cada municipi, permeten detectar padrons amb poca cobertura de dades i faciliten la comparació del volum entre Castellví de Rosanes i Sant Feliu de Llobregat al llarg dels anys disponibles.

<u>Apartat B</u> - Semblança entre els nodes.

L'objectiu d'aquest apartat és calcular la similitud entre nodes com a pas previ a la identificació d'individus que apareixen en múltiples padrons i que seran units posteriorment amb arestes *same_as*. Per fer-ho de forma completa, primer enriquim el graf amb una nova relació que connecta habitatges equivalents entre anys.

```
MATCH (h1:Habitatge)
MATCH (h2:Habitatge)
WHERE h1.llarId = h2.llarId AND h1.location = h2.location AND h1.anyPadro < h2.anyPadro
MERGE (h1)-[:MATEIX_HAB]->(h2)
RETURN count(*) AS ArestesMATEIX_HAB
```

Dos habitatges es consideren el mateix lloc físic si comparteixen *llarId* i *location*, és a dir, la mateixa llar al mateix municipi. La relació *mateix_hab* s'orienta des de l'habitatge amb l'any de padró més petit cap al més gran, garantint que no es creïn arestes duplicades en sentit invers. Aquesta relació és fonamental per al càlcul de similitud posterior, ja que permet connectar individus que han viscut al mateix habitatge en padrons d'anys diferents, cosa que d'altra manera el graf no podria capturar.

El següent pas és projectar el graf en memòria, necessari per poder treballar amb ell posteriorment.

```
CALL gds.graph.project(
  'grafSimilitud',
  ['Individu', 'Habitatge'],
  ['VIU', 'FAMILIA', 'MATEIX_HAB']
)
```

El graf inclou tres tipus de relacions per maximitzar la informació disponible per al càlcul de similitud. La relació *viu* connecta individus als seus habitatges, *família* captura les relacions familiars entre individus, i *mateix_hab* enllaça habitatges equivalents entre padrons. D'aquesta manera, dos individus podran ser considerats similars no només si viuen al mateix habitatge en el mateix padró, sinó també si viuen en habitatges equivalents en padrons d'anys diferents i comparteixen els mateixos familiars.

Finalment, tal i com demana l'exercici, calculem la similitud entre els nodes del graf que acabem de crear i escrivim el resultat de nou a la base de dades.

```
CALL gds.nodeSimilarity.write('grafSimilitud', {
  writeRelationshipType: 'SIMILAR',
  writeProperty: 'score'
})
YIELD nodesCompared, relationshipsWritten
RETURN nodesCompared, relationshipsWritten
```

L'algorisme Node Similarity calcula l'índex de Jaccard entre parells de nodes. Un score d'1 indica que dos nodes comparteixen exactament els mateixos veïns, mentre que un score proper a 0 indica poca o cap coincidència. En el context dels padrons, un score alt entre dos individus significa que comparteixen habitatge (o un d'equivalent) i els mateixos familiars, cosa que és un indicador molt fort que es tracta de la mateixa persona registrada en padrons d'anys consecutius. Els resultats s'escriuen directament a la base de dades com a relacions *similar* amb la propietat *score*, de manera que poden ser consultats posteriorment per identificar les millors candidates a ser unides amb una aresta *same_as* i completar així la traçabilitat d'individus al llarg dels anys que és l'objectiu central del projecte.

TREBALL EN EQUIP

Aquest projecte ha estat desenvolupat de forma col·laborativa entre tots quatre membres de l'equip. La metodologia de treball ha estat conjunta al llarg de totes les fases: les decisions de disseny, la implementació dels scripts Cypher i la redacció de l'informe s'han pres i elaborat de forma conjunta, discutint i validant cada pas entre tots.

Pel que fa a la gestió del repositori GitHub, ens hem repartit la responsabilitat de pujar els arxius i fer els commits de manera que tots els membres hagin contribuït de forma equitativa.

Tot i el repartiment de commits, insistim en el fet que el treball ha estat genuïnament conjunt. Cap membre ha treballat de forma aïllada en cap apartat, i totes les decisions tècniques i de disseny reflectides en aquest informe són fruit del consens de l'equip.